

Acquisition et Nettoyage des Données

Collecte par scraping web

Master 1 Data & IA — 2025-2026

Ahmad ALMAKSOUR — Faculté de Gestion, Économie et Sciences

Plan de la séance

01

Scraping
vs API

Légalité, robots.txt, matrice outils

02

HTML
& DOM

Arbre, sélecteurs CSS, XPath

03

Beautiful
Soup

Pages statiques, tables, encodages

04

Scrapy

Architecture industrielle, Spider

05

Playwright

JavaScript, headless, réseau

06

Pipeline
robuste

Pattern complet, checkpoint, prod

Pourquoi scraper ? Données sans API

La grande majorité des données sur le web n'est pas exposée via une API. Pour les atteindre, il faut les extraire directement du HTML de la page.

Pas d'API du tout

Le site ne propose aucun endpoint. Les données ne sont accessibles qu'en naviguant manuellement.

PAP.fr (annonces immo)

Pages jaunes

Annuaire publics

Archives de presse

API insuffisante

Une API existe mais ne couvre pas tous les champs, ou impose des limites trop restrictives.

Quota trop bas pour votre volume

Données historiques non exposées

Champs absents de l'API

Nécessite un abonnement payant

Données comparatives

Vous avez besoin de consolider des données de plusieurs sources hétérogènes sans API commune.

Prix concurrents (e-commerce)

Avis multi-plateformes

Données de plusieurs agences

Veille concurrentielle

Légalité et éthique du scraping

Scrapper n'est pas toujours légal ni éthique. Avant d'écrire la moindre ligne de code, trois vérifications sont obligatoires.

1	robots.txt Fichier texte à la racine de chaque site. Il indique aux robots quelles pages sont autorisées ou interdites à l'exploration.	<pre># Exemple robots.txt User-agent: * Disallow: /admin/ Disallow: /private/ Allow: /public/ Crawl-delay: 5</pre>	A retenir : Respecter Crawl-delay. Ne jamais aller sur Disallow.
2	CGU du site Les Conditions Générales d'Utilisation peuvent explicitement interdire le scraping. En France, elles ont valeur contractuelle.	<pre># Signal d'alarme dans les CGU "Toute reproduction automatisée des contenus est interdite" => Risque juridique réel</pre>	A retenir : Lire les CGU. En cas de doute, demander l'autorisation par écrit.
3	Droit à la donnée En France, la loi pour une République Numérique (2016) et le RGPD encadrent la réutilisation de données personnelles.	<pre># Données personnelles = RGPD Nom, email, téléphone... => Collecte interdite sans base légale explicite</pre>	A retenir : Ne jamais collecter de données personnelles sans base légale.

Choisir l'outil selon la complexité

Il n'existe pas un outil universel. La règle : commencer par le plus simple qui fonctionne, et monter en complexité uniquement si nécessaire.

requests + regex	requests + BS4	Scrapy	Playwright / Selenium
Cas d'usage	Cas d'usage	Cas d'usage	Cas d'usage
Page HTML simple Une seule URL Données tabulaires	Navigation DOM Multiples pages Tables et listes	Centaines de pages Spider structuré Pipeline de sortie	SPA React/Vue Contenu JS dynamique Interactions (clic, scroll)
Limites	Limites	Limites	Limites
Fragile sur HTML complexe Zéro JS	Pas de JS Pas de session complexe	Courbe d'apprentissage Overkill pour 1 page	Lent (vrai navigateur) Détecté plus facilement

Politeness - Scraper sans nuire au serveur

Un scraper mal configuré peut surcharger un serveur et provoquer un déni de service involontaire. La politeness n'est pas optionnelle.

Scraper agressif — interdit

```
# 1000 requetes en 10 secondes
for url in urls:
    r = requests.get(url) # sans pause
# 100 req/s -> DDoS involontaire
# Blocage IP garanti
# Risque juridique (CFAA en US)
```

Pause aléatoire

`random.uniform(1, 3)` simule un comportement humain. Une pause fixe de 1s est trop régulière pour paraître humaine.

Crawl-delay

Respecter la valeur indiquée dans `robots.txt`. Si absente, 1-3 secondes est une valeur raisonnable.

Scraper poli — correct

```
# Memes 1000 URLs, scraping respectueux
import time, random
for url in urls:
    r = session.get(url, timeout=15)
    r.raise_for_status()
    delay = random.uniform(1.5, 3.5)
    time.sleep(delay) # pause aleatoire
```

User-Agent honnête

Identifier votre bot clairement ('MyCrawler/1.0 contact@...'). Ne pas usurper l'identité d'un navigateur commercial.

Heures creuses

Planifier les crawls lourds la nuit ou le week-end pour minimiser l'impact sur les utilisateurs réels.

Scraper ou appeler une API ? Analyse de 4 cas réels

Pour chaque situation, analysez la décision technique et ses implications éthiques, juridiques et opérationnelles.

Cas 1

PAP.fr — Prix des appartements à Lille

L'agence immobilière veut une veille hebdomadaire des prix. PAP.fr n'a pas d'API publique. Les CGU interdisent le scraping commercial. Les données affichées sont publiques.

Cas 2

data.gouv.fr — Nouvelles publications

Un pipeline doit détecter automatiquement quand un nouveau dataset DVF est publié sur data.gouv.fr.

Cas 3

Pages jaunes — Coordonnées d'entreprises

Vous devez constituer un fichier de prospects pour une campagne B2B. Pages Jaunes n'a pas d'API publique.

RGPD critique : collecte de données personnelles (nom, téléphone) sans base légale. Risque pénal. Alternative légale : acheter une base de données qualifiée auprès d'un fournisseur agréant avec consentement.

Cas 4

Leboncoin — Prix de revente de véhicules

Un assureur veut estimer la valeur marchande de véhicules. Leboncoin n'expose pas d'API. Les CGU sont ambiguës.

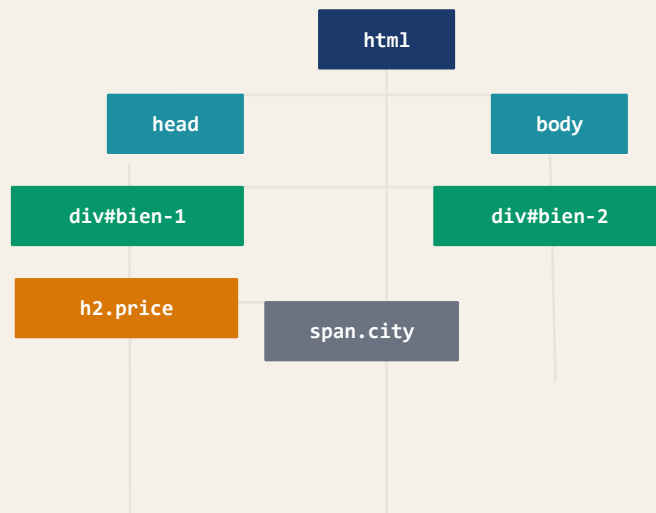
Cas borderline. La Cour de justice de l'UE (2021) a reconnu la licéité du scraping de données publiques non personnelles à des fins d'intérêt public. Consulter un juriste, documenter la démarche, anonymiser.

Anatomie d'une page HTML - Ce que voit le scraper

Avant de scraper quoi que ce soit, comprendre comment le navigateur reçoit et interprète le HTML est indispensable.

```
<!DOCTYPE html>
<html lang='fr'>
<head>
  <title>Annonces DVF</title>
</head>
<body>
  <div class='property' id='bien-1'>
    <h2 class='price'>245 000 EUR</h2>
    <span class='city'>Lille</span>
    <p class='surface'>65 m2</p>
    <a href='/bien/1'>Voir</a>
  </div>
  <div class='property' id='bien-2'>
    <h2 class='price'>189 000 EUR</h2>
    <span class='city'>Roubaix</span>
  </div>
</body>
</html>
```

Arbre DOM résultant



Le scraper navigue dans cet arbre pour trouver les nœuds cibles, puis extrait leur texte ou leurs attributs.

Sélecteurs CSS - Cibler les éléments du DOM

Les sélecteurs CSS sont la syntaxe universelle pour cibler des noeuds dans le DOM. BeautifulSoup et Playwright les supportent tous les deux.

Sélecteur	Syntaxe CSS	Exemple HTML	Trouve...
Balise	<code>div</code>	<code><div>...</div></code>	Tous les div
Classe	<code>.price</code>	<code><h2 class='price'></code>	Éléments avec class=price
ID	<code>#bien-1</code>	<code><div id='bien-1'></code>	L'élément avec id=bien-1 (unique)
Classe + balise	<code>h2.price</code>	<code><h2 class='price'></code>	Les <h2> ayant class=price
Attribut	<code>[data-id]</code>	<code><div data-id='42'></code>	Éléments avec l'attribut data-id
Attribut = valeur	<code>[data-type='apt']</code>	<code><div data-type='apt'></code>	Éléments où data-type='apt'
Descendant	<code>div.property p</code>	<code><div><p>txt</p></div></code>	Tout <p> dans un div.property
Enfant direct	<code>ul > li</code>	<code>item</code>	Seulement les enfants directs de
Pseudo :nth	<code>tr:nth-child(2)</code>	<code><tr>...</tr></code>	La 2e ligne d'un tableau
Combiné	<code>div.prop > h2.price</code>	<code><h2 class='price'></code>	h2.price enfant direct de div.prop

DevTools du navigateur - Trouver les bons sélecteurs

Avant d'écrire du code, les DevTools du navigateur permettent d'inspecter le DOM et de tester les sélecteurs en direct.

1

F12 ou Clic droit => Inspecter

Ouvre les DevTools sur l'élément ciblé. Le DOM est affiché à droite, surligné en temps réel.

2

Onglet Elements

Naviguez dans l'arbre HTML. Survolez un noeud pour le voir s'illuminer dans la page.

3

Console → \$\$ (selector)

Testez votre sélecteur CSS directement. `$('.h2.price')` retourne le premier match, `$$('h2.price')` retourne tous.

4

Clic droit sur l'élément → Copy → Copy selector

Génère le sélecteur CSS automatiquement. Attention : souvent trop spécifique (`#content > div:nth-child(3)...`).

5

Network → XHR/Fetch

Vérifiez si les données arrivent via une requête API cachée. Si oui, appeler l'API directement est plus simple que scraper le HTML.

La requête réseau cachée - Chercher l'API avant de scraper

Beaucoup de sites chargent leurs données via des appels API JSON invisibles dans le HTML. Les trouver est toujours plus simple que scraper le rendu.

Inspecter l'onglet Network → Filtrer XHR/Fetch → Chercher les appels JSON

API JSON cachée trouvée

```
# URL trouvée dans Network/XHR
# ex: https://site.fr/api/listings
r = session.get(
    'https://site.fr/api/listings',
    params={'city': 'Lille', 'limit': 100}
)
# JSON propre, structure stable
data = r.json()['results']
```

Avantages : pas de parsing HTML, structure stable, souvent paginée proprement, 10x moins de code.

Pas d'API — scraping HTML nécessaire

```
from bs4 import BeautifulSoup
import requests

r = session.get('https://site.fr/annonces')
soup = BeautifulSoup(r.text, 'lxml')
items = soup.select('div.listing')
for item in items:
    price = item.select_one('.price').text
```

Inconvénients : fragile aux changements de design, nécessite maintenance, peut être bloqué par anti-bot.

BeautifulSoup - Le duo requests + BS4

BeautifulSoup transforme une chaîne HTML brute en un arbre d'objets Python navigables. C'est l'outil idéal pour les pages statiques.

```
import requests
from bs4 import BeautifulSoup

# 1. Récupérer le HTML
session = requests.Session()
session.headers['User-Agent'] = 'MonScraper/1.0 (contact@ucl.fr)'

r = session.get('https://example.fr/annonces', timeout=15)
r.raise_for_status()
# Forcer l'encodage si nécessaire (fréquent sur les vieux sites FR)
r.encoding = r.apparent_encoding

# 2. Parser le HTML
soup = BeautifulSoup(r.text, 'lxml')

# 3. Naviguer dans le DOM
titre = soup.find('h1').get_text(strip=True)
annonces = soup.find_all('article', class_='listing')
print(f'{len(annonces)} annonces trouvees')

# 4. Extraire des données
for a in annonces[:3]:
    print(a.select_one('.price').get_text(strip=True))
```

`find(tag, attrs)`

Retourne le 1er élément correspondant. None si absent.
`find('h2', class_='price')`

`find_all(tag)`

Retourne une liste de tous les éléments. `find_all('div', class_='prop')`

`select(css)`

Sélecteur CSS complet. Retourne une liste.
`soup.select('div.prop > h2')`

`select_one(css)`

Sélecteur CSS, retourne le 1er ou None. Équivalent de `find()` mais syntaxe CSS.

`.get_text()`

Texte brut de l'élément sans les balises HTML. `strip=True` retire les espaces.

`.get('href')`

Lit la valeur d'un attribut. `tag.get('href')`, `tag['class']`, `tag['data-id']`

Extraire un tableau HTML vers un DataFrame Pandas

Les tableaux HTML <table> sont parmi les cibles les plus courantes. Pandas offre un raccourci.

Méthode 1 — `pd.read_html()` (quand ca marche)

```
import pandas as pd

# Pandas detecte et parse automatiquement
# tous les <table> d'une URL
tables = pd.read_html(url)
df = tables[0] # le premier tableau
print(df.head())

# Avec authentication/session
from io import StringIO
html_text = session.get(url).text
tables = pd.read_html(StringIO(html_text))
df = tables[0]
```

Méthode 2 — BS4 manuelle (robuste)

```
table = soup.find('table', id='prix-communes')
rows, headers = [], []

# Entêtes
for th in table.find_all('th'):
    headers.append(th.get_text(strip=True))

# Lignes de données
for tr in table.find('tbody').find_all('tr'):
    cells = tr.find_all(['td', 'th'])
    row = [c.get_text(strip=True) for c in cells]
    rows.append(row)

df = pd.DataFrame(rows, columns=headers)

# Nettoyage des types
df['Prix'] = (df['Prix']
              .str.replace('EUR', '')
              .str.replace(' ', '')
              .astype(float))
```

Robustesse - Erreurs, encodages et données manquantes

Les pages web réelles sont imparfaites. Un scraper de production doit gérer gracieusement les absences, les encodages et les structures inattendues.

```
def extraire_annonce(article) -> dict | None:
    """Extrait les données d'une annonce, retourne None si invalide."""
    try:
        # Extraction sécurisée avec valeur par défaut
        prix_el = article.select_one('.price')
        prix_txt = prix_el.get_text(strip=True) if prix_el else None

        ville_el = article.select_one('.city')
        ville = ville_el.get_text(strip=True) if ville_el else 'Inconnu'

        surface_el = article.select_one('.surface')
        surface_txt = surface_el.get_text(strip=True) if surface_el else None

        # Conversion et nettoyage
        if prix_txt is None:
            return None # champ obligatoire absent
        prix = float(prix_txt.replace('EUR','').replace(' ', ''))

        surface = None
        if surface_txt:
            surface = float(surface_txt.replace('m2','').strip())

        return {'prix': prix, 'ville': ville, 'surface': surface}
```

Encodage mal détecté

```
r.encoding = r.apparent_encoding # utiliser
chardet
soup = BeautifulSoup(r.content, 'lxml',
from_encoding='utf-8')
```

AttributeError sur None

```
prix_el = soup.select_one('.price')
prix = prix_el.get_text() if prix_el else None
# toujours tester
```

Texte avec espaces cachés

```
texte.strip() n'enlève pas tous les espaces
texte = ' '.join(texte.split()) # normalise
tous les whitespace
```

Scraping multi-pages - Gérer la pagination

Un vrai site de données comporte des dizaines ou centaines de pages. Le scraper doit les parcourir automatiquement, en s'arrêtant quand il n'y a plus de page suivante.

```
import time, random
from urllib.parse import urljoin

BASE_URL = 'https://example.fr/annonces'

def scraper_toutes_pages(base_url: str, max_pages: int = 50) -> list[dict]:
    resultats = []
    url_courante = base_url
    page = 1

    while url_courante and page <= max_pages:
        print(f'Page {page} : {url_courante}')

        r = session.get(url_courante, timeout=15)
        r.raise_for_status()
        soup = BeautifulSoup(r.text, 'lxml')

        # Extraire les annonces de cette page
        articles = soup.select('article.listing')
        nouveaux = [extraire_annonce(a) for a in articles]
        resultats += [r for r in nouveaux if r is not None]
        print(f' -> {len(nouveaux)} annonces, total : {len(resultats)}')

        # Trouver le lien 'page suivante'
        lien_suivant = soup.select_one('a.pagination-next, [rel="next"]')
        if lien_suivant:
            url_courante = urljoin(base_url, lien_suivant['href'])
        else:
            url_courante = None    # fin de la pagination

        # Politeness
        time.sleep(random.uniform(1.5, 3.0))
        page += 1

    print(f'Scraping termine : {len(resultats)} annonces sur {page-1} pages')
    return resultats
```

Lien 'page suivante'

a[rel='next'] ou a.next-page. Le plus courant. S'arrêter quand le lien est absent.

Paramètre URL ?page=N

Incrémenter page=1, 2, 3... S'arrêter quand la page retourne 0 résultats ou 404.

Paramètre ?offset=N

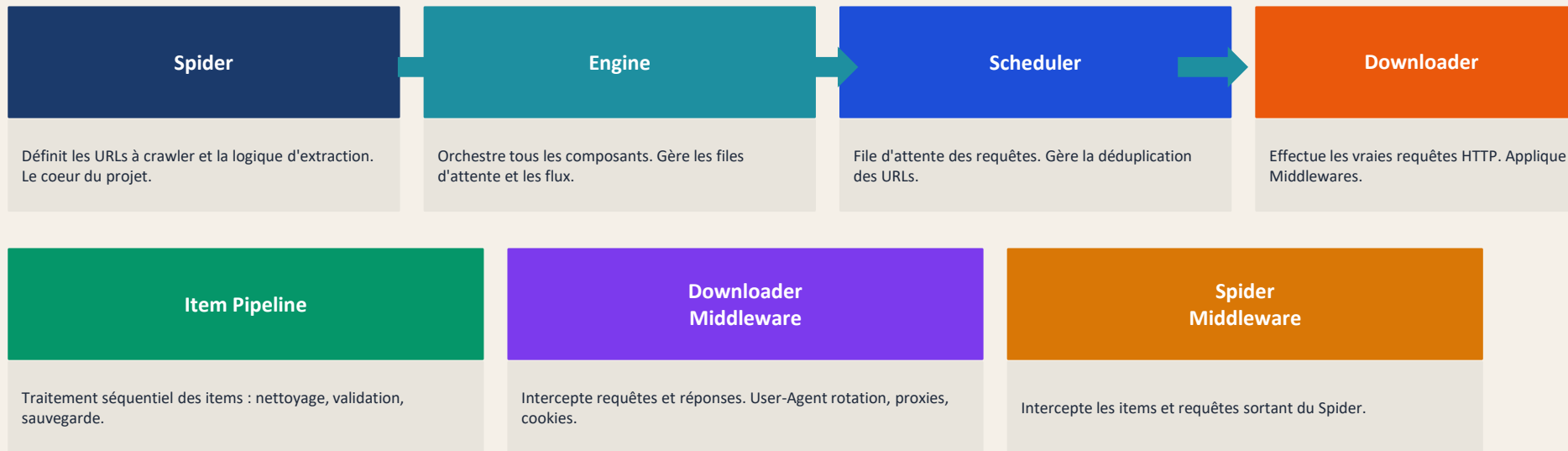
offset=0, 100, 200... Utile pour les APIs déguisées en pages HTML.

Bouton 'Charger plus'

Pas de navigation de page — contenu chargé par JavaScript. → Playwright nécessaire.

Scrapy - Architecture industrielle

Scrapy est un framework complet de scraping. Contrairement à requests+BS4, il gère nativement la concurrence, les erreurs, le rate limiting et l'export de données.



Écrire un Spider Scrapy de zéro

Un Spider est une classe Python qui définit les URLs à crawler et comment extraire les données de chaque page.

```
import scrapy
from scrapy import Field, Item
from urllib.parse import urljoin

class AnnonceDVF(Item):
    url = Field()
    prix = Field()
    ville = Field()
    surface = Field()
    type_bien = Field()

class DVFSpider(scrapy.Spider):
    name = 'dvf_spider'
    allowed_domains = ['example-immobilier.fr']
    start_urls = ['https://example-immobilier.fr/annonces']

    custom_settings = {
        'DOWNLOAD_DELAY': 2, # 2s entre requetes
        'RANDOMIZE_DOWNLOAD_DELAY': True, # +/- 50% aleatoire
        'ROBOTSTXT_OBEY': True, # respecter robots.txt
        'LOG_LEVEL': 'INFO',
    }

    def parse(self, response):
        """Parseur de la page de liste."""
        for article in response.css('article.listing'):
            prix_txt = article.css('.price::text').get(default='')
            item = AnnonceDVF(
                url = response.url,
                prix = float(prix_txt.replace('EUR', '').replace(' ', '')),
                ville = article.css('.city::text').get(),
                surface = article.css('.surface::text').get(),
                type_bien = article.css('[data-type]::attr(data-type)').get(),
            )
            yield item

        # Pagination
        next_url = response.css('a[rel="next"]::attr(href)').get()
        if next_url:
            yield scrapy.Request(urljoin(response.url, next_url), callback=self.parse)
```

::text et ::attr

Sélecteurs CSS Scrapy étendus.
article.css('.price::text').get() extrait le texte directement.

yield Request

Scrapy gère une file de requêtes asynchrones. yield Request() planifie la page suivante.

yield Item

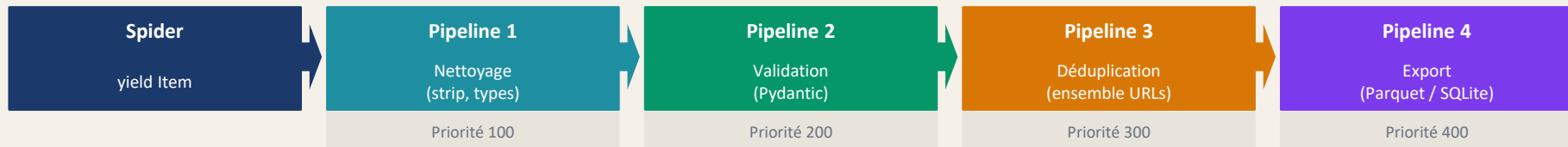
Envoie un item dans le pipeline. Scrapy le route vers les Item Pipelines configurés.

custom_settings

Surcharge la config pour ce spider. DOWNLOAD_DELAY, ROBOTSTXT_OBEY, CONCURRENT_REQUESTS...

Item Pipeline - Nettoyer, valider et exporter

Les Items sortant du Spider passent séquentiellement par des Pipelines. Chaque Pipeline a une responsabilité unique : valider, nettoyer, ou persister.



Scrapy en pratique — Commandes et configuration

Scrapy s'utilise en ligne de commande. La configuration dans settings.py centralise le comportement de tous les spiders du projet.

Commandes essentielles

```
# Créer un projet
scrapy startproject dvf_scraper

# Créer un spider
scrapy genspider dvf_spider example-immo.fr

# Tester un spider (sans exécuter)
scrapy check dvf_spider

# Lancer le spider
scrapy crawl dvf_spider

# Exporter en JSON ou CSV
scrapy crawl dvf_spider -o data.json
scrapy crawl dvf_spider -o data.csv

# Shell interactif (debug)
scrapy shell 'https://example-immo.fr'
>>> response.css('article.listing')
```

settings.py — configuration globale

```
# settings.py
BOT_NAME = 'dvf_scraper'
ROBOTSTXT_OBEY = True
DOWNLOAD_DELAY = 2
RANDOMIZE_DOWNLOAD_DELAY = True
CONCURRENT_REQUESTS = 4
AUTOTHROTTLING_ENABLED = True
AUTOTHROTTLING_TARGET_CONCURRENCY = 2.0

DEFAULT_REQUEST_HEADERS = {
    'User-Agent': 'DVFScraper/1.0 (contact@ucl.fr)',
    'Accept-Language': 'fr-FR',
}

ITEM_PIPELINES = {
    'dvf.pipelines.NettoyagePipeline' : 100,
    'dvf.pipelines.DedupPipeline' : 300,
    'dvf.pipelines.ParquetPipeline' : 400,
}
```

Discussion - Scrapy : gain réel ou complexité inutile ?

Scrapy apporte une vraie valeur ajoutée — mais à quel prix ? Analysons 4 scénarios pour décider quand l'adopter.

Scénario A

Scraper 3 pages Wikipedia pour un TP

Verdict : requests + BS4

Scrapy ici est un marteau pour enfoncer une punaise. Le setup (startproject, settings, pipeline) prend 30 min pour un résultat atteignable en 15 lignes de requests+BS4.

Scénario C

Scraping mensuel d'un rapport PDF publié par une collectivité

Verdict : requests + BS4

Un seul fichier, une seule URL, une exécution par mois. Scrapy = surcoût de maintenance injustifié. requests + téléchargement du PDF en 5 lignes.

Scénario B

Crawler 50 000 pages d'annonces immobilières

Verdict : Scrapy

À ce volume : la concurrence asynchrone de Scrapy (CONCURRENT_REQUESTS=16) est 10x plus rapide qu'une boucle synchrone. Le scheduling, la déduplication et les logs sont gérés nativement.

Scénario D

Pipeline de veille concurrentielle sur 200 sites différents

Verdict : Scrapy + middleware

200 spiders, 200 structures HTML différentes. L'architecture Scrapy (un Spider par domaine, un Pipeline commun) est la seule qui passe à l'échelle sans explosion de complexité.

Pourquoi requests ne suffit plus - Le web dynamique

Une fraction croissante du web est rendu côté client par JavaScript. requests récupère le HTML source, avant l'exécution du JS — souvent vide.

Site statique — requests fonctionne



BeautifulSoup parse le HTML → données trouvées immédiatement.

SPA / JS dynamique — requests echoue



JavaScript non executé → div vide → BS4 ne trouve rien.

Technologies qui nécessitent Playwright / Selenium

React / Vue / Angular

SPA — tout le rendu se fait côté client. Le HTML source est quasi-vide.

CAPTCHA / login requis

Playwright peut remplir des formulaires et gérer les sessions authentifiées.

Infinite scroll

Contenu chargé par scroll. requests ne peut pas déclencher l'événement scroll.

Contenu derrière un clic

Onglets, modales, boutons 'Voir plus'. Playwright peut simuler les interactions.

Playwright - Bases du browser headless

Playwright lance un vrai navigateur (Chromium, Firefox, WebKit) en mode invisible. Il exécute le JavaScript et rend la page comme un utilisateur réel.

```
from playwright.sync_api import sync_playwright
import time

with sync_playwright() as p:
    # Lancer Chromium en mode headless (invisible)
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()

    # Naviguer vers une page dynamique
    page.goto('https://example-spa.fr/annonces', timeout=30000)

    # Attendre que le contenu soit chargé
    page.wait_for_selector('article.listing', timeout=10000)

    # Récupérer le HTML après execution JS
    html = page.content()
    from bs4 import BeautifulSoup
    soup = BeautifulSoup(html, 'lxml')
    annonces = soup.select('article.listing')
    print(f'{len(annonces)} annonces trouvees')

    # Cliquer sur un bouton
    page.click('button.load-more')
    page.wait_for_selector('article.listing:nth-child(20)')

    # Remplir un formulaire
    page.fill('#search-input', 'Lille')
    page.select_option('#type-bien', 'Appartement')
    page.click('button[type="submit"]')
    page.wait_for_load_state('networkidle')

browser.close()
```

`page.goto(url)`

Navigue vers une URL. Attend que la page soit chargée.

`page.wait_for_selector()`

Attend qu'un sélecteur CSS apparaisse dans le DOM.
Timeout configurable.

`page.click(selector)`

Simule un clic. Attend que l'élément soit visible et cliquable.

`page.fill(selector, text)`

Remplit un champ input. Plus robuste que `keyboard.type()`.

`page.wait_for_load_state()`

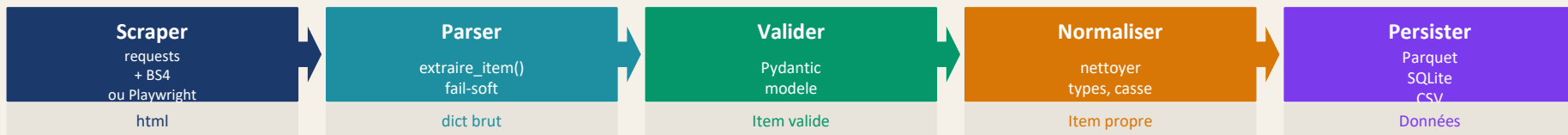
Attend un état : 'load', 'domcontentloaded', 'networkidle'.

`page.content()`

Retourne le HTML complet après exécution du JS.

Pattern pipeline - Scrape => Valide => Persiste

Un scraper de production ne se contente pas d'extraire. Il valide, normalise, gère les erreurs et sauvegarde dans un format exploitable.



Checkpoint - Reprendre un scraping interrompu

Un scraping de 5 000 URLs prend des heures. Si le script plante à mi-chemin, il faut pouvoir reprendre sans recommencer depuis zéro.

Sans checkpoint — catastrophique

```
urls_restantes = list(df_urls['url'])
for url in urls_restantes:
    data = scraper_page(url)
    # PLANTAGE à la requete 2347/5000
    # -> relancer depuis 0
    # -> 2347 requetes inutiles (39 min)
```

Plantage à la requête 2347/5000 → recommencer depuis 0 → 39 minutes perdues → nouvelles requêtes inutiles sur le serveur.

Avec checkpoint — reprise propre

```
CHECKPOINT = Path('done.json')

def charger_fait() -> set:
    if CHECKPOINT.exists():
        return set(json.loads(CHECKPOINT.read_text()))
    return set()

def marquer_fait(url: str, fait: set):
    fait.add(url)
    CHECKPOINT.write_text(json.dumps(list(fait)))

fait = charger_fait()
resultats = []
for url in urls_restantes:
    if url in fait:
        continue # deja traite
    data = scraper_page(url)
    if data:
        resultats.append(data)
        marquer_fait(url, fait)
    time.sleep(random.uniform(1.5, 3.0))

print(f'Reprise depuis {len(fait)} URLs deja traitees')
```


Checklist - Scraper de production

01

Lire robots.txt avant tout

Vérifier ROBOTSTXT_OBEY=True dans Scrapy, ou parser manuellement. Ne jamais aller dans les zones interdites.

03

Pause aléatoire obligatoire

`random.uniform(1.5, 3.0)` minimum. 5 secondes si le site est petit. Ne jamais utiliser une pause fixe — trop régulière.

05

Fail-soft sur l'extraction

`try/except` sur chaque item. Un item qui plante ne doit jamais arrêter le pipeline. Logger les rejets.

02

User-Agent honnête

Identifier votre bot (NomBot/1.0 contact@...). Ne pas usurper Chrome ou Firefox pour tromper le serveur.

04

Timeout sur chaque requête

`timeout=15` sur toutes les requêtes HTTP. Sans timeout, une requête bloque indéfiniment et plante le pipeline.

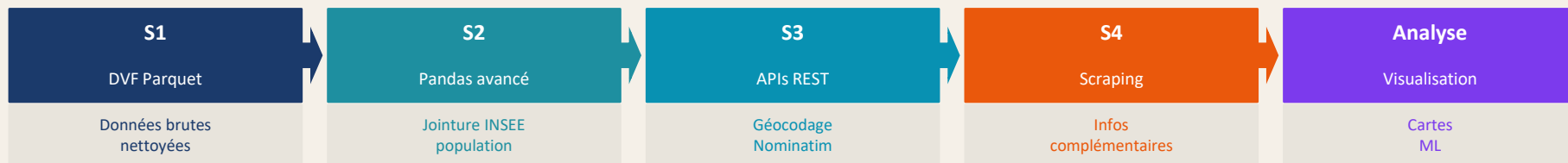
06

Checkpoint pour les longs runs

Sauvegarder l'état (URLs traitées) après chaque item. Permettre la reprise sans recommencer.

Fil rouge - Enrichir le dataset DVF par scraping

Comment le scraping s'intègre dans le pipeline DVF construit depuis la séance 1. Chaque séance ajoute une couche d'enrichissement.



Ce que le scraping peut apporter au pipeline DVF

Diagnostics de performance des communes

Certains indicateurs (taux de chômage communal, attractivité scolaire) ne sont pas dans les APIs. Des sites comme observatoire-des-territoires.gouv.fr publient des tableaux HTML.

Données des agences immobilières

Le prix affiché vs le prix DVF de vente effective. Calculer la décote moyenne par agence ou par commune.

Informations sur les logements neufs

Les permis de construire et les programmes neufs publiés par les mairies sont parfois dans des PDF ou des tableaux HTML non structurés.

Ce qu'on a vu aujourd'hui

01

Scraping vs API

Légalité, robots.txt, matrice d'outils selon complexité, politeness

03

BeautifulSoup

requests+BS4, find/select, tables→DataFrame, encodages, pagination

05

Playwright

Pages dynamiques JS, attente intelligente, interception réseau, async

02

HTML & DOM

Arbre DOM, sélecteurs CSS, XPath, DevTools, requête réseau cachée

04

Scrapy

Architecture Spider/Engine/Pipeline, settings, Shell, cas d'usage

06

Pipeline robuste

Pattern scrape→valide→persiste, checkpoint, logging, fil rouge DVF